Post-training for LLMs

CS115B – Spring 2026

James Pustejovsky

April 10, 2026

Today's place in the pipeline

- **Pretraining**: next-token prediction on large raw corpora.
- **Post-training**: modify the pretrained policy to be more *helpful*, *safe*, *steerable*, and often *better at reasoning*.
- Three big phases in today's lecture:
  1. supervised instruction tuning (SFT),
  2. preference alignment (RM, RLHF, DPO-family),
  3. test-time compute and reranking.
- Main question: *how do we change the objective from "predict web text" to "behave the way we want"?*

Why post-training exists

Base model mismatch

Pretraining optimizes

$$\max_{\theta} \sum_{t} \log \pi_\theta(x_t \mid x_{<t}),$$

but users want *instruction following*, *truthfulness*, *harmlessness*, *format control*, and often *multi-step reasoning*.

- A base model completes text; it does not automatically obey instructions.
- Prompting helps, but only up to the capabilities latent in the pretrained model.
- Post-training reshapes the model distribution $\pi_\theta(y \mid x)$ over responses y given prompt x.

From prompt completion to instruction following

Base LM view

- Continue text plausibly
- Sensitive to prompt wording
- May imitate bad internet behavior

Instruction-tuned view

- Map instruction $\rightarrow$ useful response
- Prefer assistant-style completions
- Better zero-shot generalization across tasks

$$x = \text{“Translate to French: the small dog”} \quad \rightsquigarrow \quad y = \text{“le petit chien”}$$

instead of merely continuing the prompt with another English sentence.

Supervised fine-tuning (SFT): data format

Each training example is a prompt-response pair

$$(x_i, y_i)$$

where x_i may include system instructions, user turns, tool context, demonstrations, or formatting constraints.

Typical chat serialization:

<bos><system>...<user> x_i <assistant> y_i

- The architecture is unchanged.
- The objective is still token-level cross-entropy.
- The supervision is now *task-conditioned*: there is a preferred answer.

SFT objective

For dataset $\mathcal{D}_{\mathrm{SFT}} = \{(x_i, y_i)\}_{i=1}^{N}$, maximize

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = \sum_{i=1}^{N} \log \pi_\theta(y_i \mid x_i) = \sum_{i=1}^{N} \sum_{t=1}^{|y_i|} \log \pi_\theta(y_{i,t} \mid x_i, y_{i,<t}).$$

Equivalent minimization form:

$$J_{\mathrm{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\mathrm{SFT}}}[\log \pi_\theta(y \mid x)].$$

Interpretation

We are not changing the model family; we are changing the training distribution and target continuations.

Masking: train on assistant tokens only

In chat SFT we often ignore loss on prompt tokens and optimize only the assistant span:

$$J(\theta) = -\sum_{t=1}^{T} m_t \log \pi_\theta(z_t \mid z_{<t}), \qquad m_t \in \{0, 1\}$$

where $m_t = 1$ only for assistant tokens.

- Prevents the model from wasting capacity modeling fixed prompt wrappers.
- Lets us train on multi-turn transcripts without penalizing the model for user text.
- Common implementation: labels for masked tokens are set to -100 in PyTorch cross-entropy.

What makes instruction data valuable?

1. **Coverage**: many tasks, domains, languages, and styles.
2. **Diversity**: multiple phrasings for the same task.
3. **Quality**: good outputs, not just plausible outputs.
4. **Formatting**: explicit schemas, refusal style, citation style, verbosity control.

Rule of thumb

Post-training quality is often bottlenecked more by *data curation* than by optimizer sophistication.

Where instruction data comes from

- Human-written prompts and answers
- Existing supervised NLP datasets turned into instruction format
- Synthetic data from stronger LLMs
- Distillation from expert systems / tools
- Safety-specific data: benign requests, harmful requests, jailbreak attempts, refusal exemplars

Key tension:

cheap synthetic scale vs. high-fidelity human supervision

Instruction tuning as conditional distribution shift

Before SFT, the model approximates

$$p_{\text{web}}(y \mid x).$$

After SFT, it is pushed toward

$$p_{\text{assistant}}(y \mid x),$$

where $p_{\text{assistant}}$ emphasizes direct answers, formatting, politeness, hedging when uncertain, and task completion.

- SFT does *behavior shaping*.
- It is still imitation learning.
- It cannot easily optimize subtle human preferences unless those are already encoded in demonstrations.

What SFT does *not* solve well

- Fine-grained tradeoffs such as helpfulness vs. harmlessness
- Ranking two plausible responses when both look acceptable
- Long-horizon exploration or search behavior
- Robust refusal under adversarial prompting
- Reward hacking induced by benchmark-specific exemplars

Motivation for preference alignment

We often care more about *relative preference* between outputs than exact token matching.

Preference data

Instead of a single gold answer, collect comparisons:

$$(x,\ y^+,\ y^-)$$

where for prompt x, annotators prefer y^+ over y^-.

- Easier to elicit than perfect demonstrations
- Better aligned with product questions: "which answer is better?"
- Supports training a reward model or direct preference objective

Bradley–Terry preference model

Suppose a latent reward model $r_\phi(x, y)$ scores responses. Then

$$P_\phi(y^+ \succ y^- \mid x) = \frac{\exp(r_\phi(x, y^+))}{\exp(r_\phi(x, y^+)) + \exp(r_\phi(x, y^-))} = \sigma(r_\phi(x, y^+) - r_\phi(x, y^-)).$$

- Only *differences* in reward matter.
- Rewards are identifiable only up to an additive constant.
- This is the standard pairwise-comparison model behind many alignment methods.

Reward model training loss

Given preference triples (x, y^+, y^-), maximize the log-likelihood:

$$\mathcal{L}_{\mathrm{RM}}(\phi) = \sum_{(x,y^+,y^-)} \log \sigma\big(r_\phi(x, y^+) - r_\phi(x, y^-)\big).$$

Equivalent minimization:

$$J_{\mathrm{RM}}(\phi) = -\mathbb{E}\left[\log \sigma\big(r_\phi(x, y^+) - r_\phi(x, y^-)\big)\right].$$

Interpretation

The reward model learns a scalar *utility proxy* from comparisons, not from absolute scores.

Where reward models come from architecturally

Common setup:

- Start from an SFT model or pretrained LM encoder-decoder / decoder-only stack.
- Feed the entire prompt-response sequence.
- Take final hidden state $h(x, y)$, attach a scalar head:

$$r_\phi(x, y) = w^\top h(x, y) + b.$$

- Train with the pairwise logistic loss from the previous slide.

Important

The reward model is not deployed directly to users; it is used to *shape* the policy.

RLHF pipeline at a glance

Pretrained model $\rightarrow$ SFT model $\rightarrow$ Preference data $\rightarrow$ Reward model $\rightarrow$ RL optimization

At RL time:

1. Sample response $y \sim \pi_\theta(\cdot \mid x)$
2. Score it with $r_\phi(x, y)$
3. Update policy to increase expected reward
4. Regularize toward a reference policy π_{ref}

KL-regularized RLHF objective

A standard objective is

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x,y) - \beta \mathrm{KL}(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x))\big].$$

Equivalent token-level approximation for a sampled response:

$$\hat{r}(x,y) = r_\phi(x,y) - \beta \sum_t \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}.$$

- Reward term says "be preferred".
- KL term says "do not drift too far from the reference model".

Why the KL penalty matters

Without KL regularization, the policy can exploit imperfections in the reward model:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_\theta}[r_\phi(x, y)]$$

may produce bizarre, high-reward but low-quality responses.

Role of the reference policy

π_{ref} anchors the policy to the SFT model, preserving fluency and basic task competence while still allowing improvement.

- Too small β: reward hacking, mode collapse
- Too large β: almost no behavioral change

Policy gradient view

For episodic text generation,

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta} \left[(\hat{r}(x, y) - b(x)) \nabla_\theta \log \pi_\theta(y \mid x) \right]$$

where $b(x)$ is a baseline to reduce variance.
Tokenized form:

$$\log \pi_\theta(y \mid x) = \sum_t \log \pi_\theta(y_t \mid x, y_{<t}).$$

- Same REINFORCE idea as in classical RL
- High variance motivates PPO-style stabilization

PPO surrogate objective

Given old policy $\pi_{\theta_{\text{old}}}$, define token- or sequence-level ratio

$$\rho(\theta) = \frac{\pi_\theta(y \mid x)}{\pi_{\theta_{\text{old}}}(y \mid x)}.$$

PPO maximizes the clipped surrogate:

$$L^{\text{PPO}}(\theta) = \mathbb{E}\left[\min\left(\rho(\theta)A,\ \text{clip}(\rho(\theta), 1-\epsilon, 1+\epsilon)A\right)\right].$$

- A is an advantage estimate from the reward model plus KL shaping.
- Clipping prevents unstable policy jumps.
- In language models, PPO is usually run on top of an SFT initialization.

RLHF strengths and weaknesses

Strengths

- Can optimize arbitrary scalar rewards
- Supports online data collection
- Handles multi-objective reward design

Weaknesses

- Complex and expensive
- Reward model misspecification
- Credit assignment is noisy
- Sensitive to KL coefficient, rollout mix, truncation

This motivated direct preference optimization methods that avoid an explicit RL loop.

DPO: core idea

Instead of

$$\text{preferences} \rightarrow \text{reward model} \rightarrow \text{RL policy update},$$

DPO collapses the pipeline into one supervised-like objective on preference pairs.

Idea

If the optimal RLHF policy has a closed-form relation to rewards and a reference model, then we can optimize that relation directly.

From KL-constrained RL to optimal policy

For a fixed prompt x, consider

$$\max_{\pi(\cdot|x)} \mathbb{E}_{y\sim\pi}[r(x,y)] - \beta \mathrm{KL}(\pi(\cdot \mid x) \| \pi_{\mathsf{ref}}(\cdot \mid x)).$$

The optimizer is the Boltzmann policy

$$\pi^*(y \mid x) \propto \pi_{\mathsf{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x,y)\right).$$

Thus

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathsf{ref}}(y \mid x)} + C(x)$$

for some prompt-dependent constant $C(x)$.

DPO loss

Substitute the implied reward difference into the Bradley–Terry model:

$$P(y^+ \succ y^- \mid x) = \sigma\left(\beta \log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\text{ref}}(y^+ \mid x)} - \beta \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\text{ref}}(y^- \mid x)}\right).$$

Then minimize

$$J_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y^+,y^-)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\text{ref}}(y^+ \mid x)} - \beta \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\text{ref}}(y^- \mid x)}\right)\right].$$

How to read the DPO equation

Define sequence log-probability gap

$$\Delta_\theta = \log \pi_\theta(y^+ \mid x) - \log \pi_\theta(y^- \mid x).$$

Define reference gap

$$\Delta_{\text{ref}} = \log \pi_{\text{ref}}(y^+ \mid x) - \log \pi_{\text{ref}}(y^- \mid x).$$

Then DPO prefers

$$\Delta_\theta > \Delta_{\text{ref}}.$$

- Increase probability of chosen responses
- Decrease probability of rejected responses
- But only relative to what the reference already knows

Gradient intuition for DPO

For one pair,

$$\nabla_\theta J \propto -(1-\sigma(z))\,\beta\,[\nabla_\theta \log \pi_\theta(y^+ \mid x) - \nabla_\theta \log \pi_\theta(y^- \mid x)]$$

where

$$z = \beta\Big(\log \frac{\pi_\theta(y^+|x)}{\pi_{\text{ref}}(y^+|x)} - \log \frac{\pi_\theta(y^-|x)}{\pi_{\text{ref}}(y^-|x)}\Big).$$

Takeaway

The update looks like a contrastive supervised signal over full responses.

DPO versus RLHF

	RLHF	**DPO**
Intermediate reward model	yes	no
Online rollouts needed	usually yes	no
Optimization style	RL / PPO	supervised-like
Implementation complexity	high	moderate
Can optimize arbitrary scalar reward	stronger	weaker

DPO is often easier to train and reproduce, but it still inherits the quality limits of the preference data.

Beyond DPO: a family of preference objectives

The field now includes many close relatives:

- **IPO**: changes the classification geometry / implicit preference margin
- **ORPO**: combines odds-ratio preference loss with SFT in one stage
- **KTO**: uses Kahneman-Tversky style gains/losses with binary desirability labels
- **SimPO, cDPO, robust DPO**: address overconfidence, label noise, or calibration

Pedagogical point

Most of these are variants of *contrastive policy shaping* against a reference distribution.

Outcome rewards versus process rewards

For a reasoning trace $z = (z_1, \ldots, z_T)$ and final answer a:

$$R_{\text{outcome}}(x, z, a) = \mathbf{1}[a = a^\star]$$

rewards only correctness.
A process reward model instead scores intermediate steps:

$$R_{\text{process}}(x, z) = \sum_{t=1}^{T} r_t(x, z_{\leq t}).$$

- Outcome rewards are cheap but sparse.
- Process rewards are denser but require more annotation or synthetic supervision.

Group-relative policy optimization (GRPO): intuition

A prompt x generates K sampled completions

$$y^{(1)}, \ldots, y^{(K)} \sim \pi_\theta(\cdot \mid x).$$

Each completion gets score $R^{(k)}$. Use a *relative* advantage inside the group:

$$A^{(k)} = \frac{R^{(k)} - \mu_R}{\sigma_R + \epsilon} \quad \text{or} \quad A^{(k)} = R^{(k)} - \frac{1}{K} \sum_j R^{(j)}.$$

Idea

Improve the samples that were better than the prompt-local average, suppress the worse ones.

A GRPO-style objective

A simplified clipped objective mirrors PPO but with group-relative advantage:

$$L_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{K}\sum_{k=1}^{K} \min\left(\rho_k(\theta)A^{(k)}, \text{clip}(\rho_k(\theta), 1-\epsilon, 1+\epsilon)A^{(k)}\right)\right] - \beta \text{KL}(\pi_\theta \| \pi_{\text{ref}}).$$

$$\rho_k(\theta) = \frac{\pi_\theta(y^{(k)} \mid x)}{\pi_{\theta_{\text{old}}}(y^{(k)} \mid x)}.$$

This avoids a separately trained scalar value model in some implementations.

RLAIF and Constitutional AI

Human preferences are expensive. Alternative:

1. Write a constitution or rubric
2. Use a stronger model to critique / revise outputs
3. Train on AI-generated preferences or revised targets

$$\text{policy} \rightarrow \text{critique} \rightarrow \text{revision} \rightarrow \text{new training signal}$$

This can scale safety and style alignment beyond pure human labeling.

Constitutional training as self-revision

A simplified two-stage recipe:

$$
\begin{aligned}
y_0 &\sim \pi_\theta(\cdot \mid x) \\
c &= \text{Critique}(x, y_0; \mathcal{C}) \\
y_1 &= \text{Revise}(x, y_0, c; \mathcal{C})
\end{aligned}
$$

where $\mathcal{C}$ is a set of principles.
Then train on revised outputs y_1 or use pairwise preferences $y_1 \succ y_0$.

Key idea

Turn normative principles into synthetic supervision.

Alignment is multi-objective optimization

We rarely optimize one scalar notion of "good".
A useful conceptual objective is

$$R(x, y) = \lambda_h R_{\text{help}}(x, y) + \lambda_s R_{\text{safe}}(x, y) + \lambda_t R_{\text{truth}}(x, y) + \lambda_f R_{\text{format}}(x, y).$$

- The λ's change by application.
- Product behavior depends heavily on how these objectives are weighted.
- Safety failures often arise from misspecified tradeoffs, not only from model capability.

Failure modes in alignment

- **Reward hacking**: exploit reward model quirks
- **Over-refusal**: safe but unhelpful
- **Sycophancy**: agree with the user rather than with evidence
- **Verbosity bias**: longer answers may look better to raters
- **Distribution shift**: preference data does not cover deployment prompts

One deep lesson

"Aligned to the data pipeline" is not the same as "aligned to the real world".

Training-time compute versus test-time compute

Two ways to improve quality:

1. make the model better *before* inference,
2. spend more compute *during* inference.

Formalize a budgeted decision problem:

$$\max_{\text{inference policy}} \mathbb{E}[\text{utility}] \quad \text{s.t.} \quad \mathbb{E}[\text{tokens} + \text{tool calls} + \text{latency}] \leq B.$$

Post-training increasingly teaches models how to use extra inference budget well.

Chain-of-thought as latent computation

A reasoning trace z can be treated as latent or explicit computation:

$$p(a \mid x) = \sum_z p(a, z \mid x) = \sum_z p(z \mid x) p(a \mid x, z).$$

Approximations:

- sample one z (single chain)
- sample many z's and marginalize approximately
- search over z with a verifier

Self-consistency

Sample K chains of thought:

$$z^{(1)}, \ldots, z^{(K)} \sim p_\theta(z \mid x), \qquad a^{(k)} \sim p_\theta(a \mid x, z^{(k)}).$$

Then aggregate, often by majority vote:

$$\hat{a} = \arg\max_a \sum_{k=1}^{K} \mathbf{1}[a^{(k)} = a].$$

Why it helps

If multiple reasoning paths independently converge to the same answer, aggregation reduces variance.

Best-of-N sampling and reranking

Generate N candidates:

$$y^{(1)}, \dots, y^{(N)} \sim \pi_\theta(\cdot \mid x).$$

Choose

$$\hat{y} = \arg\max_{k \le N} s(x, y^{(k)}),$$

where s may come from:

- a reward model,
- a verifier,
- a classifier for correctness / safety / style,
- a committee or ensemble.

This is often the simplest effective form of test-time compute.

Verifier-guided decoding

Suppose a verifier estimates

$$v_\psi(x, y) \approx P(\text{response is correct} \mid x, y).$$

Then we can decode with combined score

$$s(x, y) = \alpha \log \pi_\theta(y \mid x) + (1 - \alpha) \log v_\psi(x, y).$$

- Language model ensures fluency and plausibility.
- Verifier pushes toward correctness.
- Common in math and code settings with executable checks.

Search over reasoning trajectories

Instead of sampling full solutions independently, do incremental search over partial traces $z_{\leq t}$.
A generic objective:

$$\hat{z} = \arg\max_{z \in \mathcal{T}(x)} \left[\log p_\theta(z \mid x) + \lambda V_\psi(x, z) \right]$$

where $\mathcal{T}(x)$ is the search tree and V_ψ estimates downstream value.
Methods:

- beam search over reasoning traces
- tree search / MCTS-like expansions
- branch-and-bound with tool feedback

Test-time compute changes the training target

If deployment uses reranking or search, the model need not place all probability mass on the single best trajectory.
Instead, a useful training target may be:

produce a candidate set containing a high-quality solution.

Implication

Post-training can optimize for *sample quality under selection*, not only one-shot greedy quality.

Parameter-efficient post-training

Often we do not update all parameters. With LoRA, a weight matrix update is

$$W' = W + \Delta W, \qquad \Delta W = BA,$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$, and $r \ll d$.

- Train only A, B, freeze W
- Memory and optimizer state scale with r, not with full model size
- Used for SFT and often for preference tuning as well

Why LoRA works surprisingly well

A full update ΔW is high-dimensional, but task adaptation may lie in a low-rank subspace. Parameter count comparison:

$$\underbrace{d_{\text{out}} d_{\text{in}}}_{\text{full finetune}} \quad \text{vs.} \quad \underbrace{r(d_{\text{out}} + d_{\text{in}})}_{\text{LoRA}}.$$

For $r \ll \min(d_{\text{in}}, d_{\text{out}})$, this is a large reduction.

Quantized post-training (QLoRA intuition)

Store pretrained weights in low precision, e.g. 4-bit, while training LoRA adapters in higher precision.
Conceptually:

$$W_{\text{fp16}} \approx \text{dequant}(W_{\text{4bit}})$$

during forward pass, while only adapter weights receive gradients.

- Makes SFT / DPO feasible on much smaller hardware
- Does not remove the need for high-quality data
- Often shifts the bottleneck from memory to data and experimentation time

Preference data pipeline in practice

1. Collect prompts from real or synthetic traffic
2. Sample multiple responses from current policy
3. Filter for toxicity / formatting / privacy
4. Ask raters or judges to compare responses
5. Train RM or DPO-style objective
6. Evaluate, red-team, refresh data

Online loop

Deployment changes the prompt distribution, so post-training is not a one-shot process.

Evaluation after post-training

There is no single metric.
Common buckets:

- task quality / win rate
- instruction following
- safety and jailbreak resistance
- factuality / hallucination rate
- reasoning accuracy on held-out tasks
- style adherence and latency

$$\text{good post-training} \neq \text{high reward-model score only}$$

Offline metrics can mislead

Suppose $s(x, y)$ is the metric used in development. Then optimizing

$$\max_{\theta} \mathbb{E}[s(x, y)], \qquad y \sim \pi_\theta(\cdot \mid x)$$

may overfit quirks of s, especially if s is itself model-based.

- Benchmark saturation
- evaluator overfitting
- contamination
- poor correlation with user satisfaction

Safety post-training and refusal style

Safety alignment is not just binary refusal.
A more realistic target is a policy that maps

$$x \mapsto \begin{cases} \text{useful answer} & \text{if request is allowed} \\ \text{brief safe refusal / redirect} & \text{if request is unsafe} \end{cases}$$

- Need both benign and adversarial data
- Need calibrated refusal boundaries
- Need style consistency under pressure / jailbreaking

Serving tradeoffs: alignment tax and product constraints

Post-training can change:

- latency,
- verbosity,
- tool-use frequency,
- refusal rate,
- answer calibration.

Useful deployment objective:

$$\text{Utility} = \text{quality} - \lambda_1 \text{latency} - \lambda_2 \text{unsafe rate} - \lambda_3 \text{user frustration}.$$

The best research model is not automatically the best product model.

Worked example: SFT loss on one chat example

Suppose prompt x is fixed and target response tokens are y_1, y_2, y_3.
Then

$$\log \pi_\theta(y \mid x) = \log \pi_\theta(y_1 \mid x) + \log \pi_\theta(y_2 \mid x, y_1) + \log \pi_\theta(y_3 \mid x, y_{1:2}).$$

If the model probabilities assigned are $0.7, 0.5, 0.2$, then

$$-\log \pi_\theta(y \mid x) = -\log(0.7) - \log(0.5) - \log(0.2).$$

Low probability on any one token raises the sequence loss sharply.

Worked example: reward model preference probability

Suppose for prompt x,

$$r_\phi(x, y^+) = 2.2, \qquad r_\phi(x, y^-) = 1.0.$$

Then

$$P(y^+ \succ y^- \mid x) = \sigma(2.2 - 1.0) = \sigma(1.2) \approx 0.77.$$

If we instead score $r_\phi(x, y^+) = 2.2 + c$ and $r_\phi(x, y^-) = 1.0 + c$, the probability is unchanged.

Lesson

Only reward *differences* matter in pairwise preference models.

Worked example: DPO on a single pair

Assume

$$\log \pi_\theta(y^+ \mid x) = -3, \quad \log \pi_\theta(y^- \mid x) = -5,$$
$$\log \pi_{\text{ref}}(y^+ \mid x) = -2.5, \quad \log \pi_{\text{ref}}(y^- \mid x) = -4.8, \qquad \beta = 1.$$

Then

$$z = (-3 + 4.8) - (-5 + 2.5) = -0.7.$$

So

$$P_\theta(y^+ \succ y^- \mid x) = \sigma(-0.7) \approx 0.33.$$

The DPO update will increase probability of y^+ relative to y^-.

Worked example: best-of-N can beat greedy

Suppose a one-shot solver gets the correct answer with probability $p = 0.35$, and the verifier can perfectly identify a correct candidate if one appears.
Then with N independent samples:

$$P(\text{at least one correct}) = 1 - (1 - p)^N.$$

For $N = 5$:

$$1 - (0.65)^5 \approx 0.884.$$

Takeaway

When verification is easier than generation, sampling plus selection can dramatically improve performance.

A modern post-training stack

One common recipe:

1. Pretrain base model
2. SFT on broad instruction data
3. Safety / style SFT refresh
4. Preference optimization (DPO / RLHF / variants)
5. Domain-specific post-training (code, math, tool use, enterprise style)
6. Test-time compute: reranking, search, verifier use
7. Continuous eval and red-teaming

Comparing the main methods

Method	Supervision	Main loss	Strength	Weakness
SFT	gold responses	token NLL	simple, stable	imitation only
RM	preference pairs	pairwise logistic	reusable scorer	misspecification
RLHF/PPO	RM scores	policy optimization	flexible objective	expensive/unstab
DPO	preference pairs	contrastive logistic	simpler than RLHF	tied to pair data
GRPO-style	sampled groups + rewards	relative policy objective	good for reasoning loops	depends on cand date pool

A unifying view

Most post-training methods adjust

$$\pi_\theta(y \mid x)$$

by combining three ingredients:

1. **supervision source**: demonstrations, preferences, verifiers, constitutions
2. **optimization rule**: NLL, pairwise logistic, policy gradient, search
3. **regularization anchor**: reference policy, KL, data mixing, safety constraints

post-training = policy shaping under imperfect supervision

Open research questions

- How do we align for truthfulness, not just preference?
- When should we optimize outcomes versus processes?
- How should training account for heavy test-time compute?
- Can model-based judges be trusted without self-reinforcing bias?
- How do we preserve helpfulness under strong adversarial pressure?
- What is the right objective when the deployment distribution keeps changing?

Take-home summary

- **SFT** teaches the model to act like an assistant.
- **Preference alignment** teaches the model which of two plausible answers is better.
- **RLHF** uses a learned reward and online policy optimization.
- **DPO-family methods** optimize preference behavior directly against a reference model.
- **Reasoning post-training** often benefits from group-relative updates, verifiers, and search.
- **Test-time compute** is now part of the capability story, not just a deployment detail.

Selected references

- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, Ch. 9 draft (2026): Post-training: Instruction Tuning, Alignment, and Test-Time Compute.
- Ouyang et al. (2022). Training language models to follow instructions with human feedback.
- Rafailov et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model.
- Wang et al. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models.
- Bai et al. (2022). Constitutional AI: Harmlessness from AI Feedback.
- Hu et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models.
- Schulman et al. (2017). Proximal Policy Optimization Algorithms.